

RedeemLoop Protocol / 兑环协议

RedeemLoop Protocol / 兑环协议

FT-first RWA 提货券与兑换券开源协议白皮书

版本: v0.1 Draft

日期: 2026-06-17

定位: 面向厂商、电商、实体店、小程序、直播带货和广告分发渠道的一键植入式链上提货券系统

状态: 产品与工程设计稿, 可交给开发团队或 Codex 进入原型开发

本文不是法律、财务、税务或投资意见。RedeemLoop 的产品原则是: 券不是积分, 不是储值余额, 不是投资品, 而是商品或服务的完整兑换权益。

目录

摘要

为什么主介质应当是 FT, NFT 只是可选分支

核心定义

系统目标与非目标

代币形态: ERC-20、ERC-6909、ERC-1155、NFT

生命周期: 核销不等于销毁

架构总览

一键代码植入: 官网、电商、小程序、直播带货、广告渠道

电商与实体店验券

Coca-Cola 型永续流转案例

链上合约设计

链下服务设计

数据模型与事件模型

用户体验

安全设计

积分分支: 只能作为旁路, 不进入主体验

合规与商业边界

MVP 开发路线

Codex 施工原则

参考资料

1. 摘要

RedeemLoop Protocol / 兑环协议是一个开源的 FT-first RWA 提货券系统。它允许厂商、品牌、电商店铺、实体零售商、软件、小程序、直播带货和广告投放渠道用一段嵌入代码发行、分发、验收和回收链上兑换券。

RedeemLoop 的核心不是把商品做成收藏品 NFT, 也不是把积分上链。它把一个清晰的商业承诺做成链上可持有、可转移、可验收的 FT 或半同质化 token:

1 个 Voucher Token = 1 个明确商品或服务的完整兑换权益

用户持有 token 后, 可以在厂商官网、电商 checkout、线下 POS、合作门店或活动现场直接兑换商品或服务。兑换时可以选择销毁该 token, 也可以把 token 转回厂商或门店金库, 由厂商再次出售、空投、广告分发或渠道投放。这样, 提货券不仅可以完成一次性核销, 也可以成为厂商可循环利用的「固定总量权益凭证」。

RedeemLoop 的核心判断是:

商品通常是同质化的, 所以主介质应该是 FT。

NFT 适合限量收藏、序列号、高价值单品、门票座位、艺术联名, 但不应是默认介质。

核销是业务事件, 不必等于销毁 token。

若厂商希望总量恒定并循环分发, 就应支持 collect-to-recirculate: 用户兑换后 token 回到厂商金库, 可二次分发。

转让和二级流转不需要协议自己重做, 应尽量使用现有钱包、转账、市场、链上基础设施。

最关键的产品能力不是炫酷链上资产, 而是任何官网、商城、小程序、广告页和收银系统都能一键植入。

2. 为什么主介质应当是 FT, NFT 只是可选分支

上一代 RWA 或 NFT 提货券方案容易犯一个错误: 把每张券做成 NFT。这样确实有收藏展示价值, 但对大多数消费品和标准化服务而言, NFT 是笨重的。

一瓶可乐、一箱水、一杯咖啡、一次洗车、一个标准 SaaS 套餐月度权益、一张通用兑换票, 本质上都是同质化权益。用户不关心自己拿到的是第 173 号还是第 98123 号, 他只关心这张券能不能兑换同一个商品。

因此 RedeemLoop 采用以下优先级:

第一优先级: FT Voucher

第二优先级: Multi-FT Voucher, 例如 ERC-6909 或 ERC-1155 的同质化 tokenId

第三优先级: SFT, 适合批次、地区、有效期、套餐

第四优先级: NFT, 仅用于唯一资产、收藏品、序列号权益或联名权益

FT 的优势:

语义简单: 每个 token 完全相同。

用户容易理解: 1 个 token 兑换 1 个商品。

链上基础设施成熟: 钱包、转账、allowance、签名授权、DEX/市场基础设施已有。

厂商易发行: 一个 SKU 一个 ERC-20, 或一个合约管理多个 FT tokenId。

便于总量恒定: 固定供应量的 token 可以在用户和厂商金库之间循环。

便于实体店验券: 收银机只需要检查余额并收取指定数量。

NFT 的正确位置:

限量收藏版权益。

每个商品有唯一编号或防伪编号。

门票座位、酒店房晚、设备序列号、保修合同。

品牌联名、有头像、有艺术图、有收藏属性。

NFT 不应该是普通可乐券、咖啡券、饮料券、通用兑换券的默认载体。

3. 核心定义

3.1 Voucher Token

Voucher Token 是链上的兑换权益凭证。它不是积分, 不是余额, 不是折扣券。一个 Voucher Token 对应一份完整权益。

1 RDL-COKE = 兑换 1 瓶可口可乐指定范围内任意型号

1 RDL-COFFEE = 兑换 1 杯指定咖啡

1 RDL-WASH = 兑换 1 次标准洗车服务

1 RDL-SAAS-MONTH = 兑换 1 个月指定软件服务

3.2 Terms / 权益条款

每一种 Voucher Token 必须绑定一个权益条款。条款需要回答：

兑换什么商品或服务。

兑换数量是多少。

哪些门店、国家、地区或渠道可用。

是否包邮、含税、含服务费。

有效期或是否永续。

是否可转让。

核销后是销毁还是回收到厂商金库。

缺货、停售、退款、换货如何处理。

Terms 的公开版本可以放在 IPFS、Arweave、厂商 CDN 或 GitHub。链上至少保存 termsHash 和 termsURI。

3.3 Merchant Vault / 厂商金库

Merchant Vault 是厂商、区域代理、渠道商或门店用来接收已兑换 token 的地址或智能合约。若 token 的 redemption mode 是 COLLECT, 则用户兑换后 token 不销毁, 而是进入 Merchant Vault。

Merchant Vault 可以再次把 token 用于：

重新销售。

空投给用户。

发给广告渠道。

发给直播间观众。

交给分销商。

门店促销。

3.4 Redemption / 兑换或核销

Redemption 是业务事件, 表示某个用户使用 token 获得了商品或服务。它可以触发不同链上结算动作：

BURN: token 被销毁。

COLLECT: token 被转入厂商金库。

LOCK_AND_FULFILL: token 先锁定, 履约完成后再销毁或收回。

3.5 Store Terminal / 门店终端

Store Terminal 是被厂商授权的验券终端。它可能是实体收银机、移动 POS、店员手机、网页后台、自动售货机、酒店前台系统或小程序收银台。

3.6 Campaign / 分发活动

Campaign 是一次发券、卖券、广告投放、直播带货或渠道促销活动。RedeemLoop 不要求厂商每次发行新 token, 厂商可以用同一批固定供应量 token 在不同 Campaign 中循环分发。

4. 系统目标与非目标

4.1 目标

RedeemLoop 的目标是让厂商拥有一种链上原生的提货券基础设施:

任何标准商品或服务都可以发行 FT 兑换券。

厂商官网可以用一段 JS 代码植入买券、领券、验券、兑换按钮。

电商系统可以把 token 当作「完整权益支付方式」, 创建 0 元订单。

实体店可以扫码验券并把 token 收回或销毁。

直播带货、广告页、小程序可以直接分发 claim link。

用户可以在现有钱包和加密基础设施里转移 token。

厂商可以选择一次性销毁模型, 也可以选择总量恒定的回收再分发模型。

NFT 只作为可选的联名、收藏或唯一资产分支。

4.2 非目标

RedeemLoop 不做以下事情:

不把券包装成投资品。

不做 K 线、涨幅榜、收益率排行。

不默认提供二级市场, 二级流转尽量使用现有钱包和市场基础设施。

不把积分伪装成提货券。

不鼓励用户先用积分再补钱的复杂抵扣体验。

不在链上存储姓名、电话、地址等 PII。

不承担所有司法区的合规解释, 只提供技术边界和合规接口。

5. 代币形态: ERC-20、ERC-6909、ERC-1155、NFT

RedeemLoop 采用可插拔 token profile。不同厂商可以按商品性质选择实现。

5.1 Profile A: ERC-20 Single Voucher Class

适合一个合约对应一种标准兑换权益。

示例:

合约名: CocaColaBottleVoucher
Symbol: COKE1
Decimals: 0
Max Supply: 100,000,000
权益: 1 token = 1 瓶指定范围内可口可乐饮品
核销策略: COLLECT
供应策略: fixed cap, no future mint after seal

特点:

最容易接入现有钱包和基础设施。

对同质化商品最自然。

每个 SKU 或权益一份合约, 语义清晰。

可以用 decimals = 0 强化「一枚 token 是一份完整权益」。

可加入 EIP-712 / EIP-3009 风格签名授权, 让用户不需要 gas 即可被门店收券。

适用:

可乐券、咖啡券、饮料券、餐券、洗车券、SaaS 月卡、标准服务券。

5.2 Profile B: ERC-6909 Multi-FT Voucher

ERC-6909 是更轻量的多 token 规格, 一个合约可以通过 id 管理多个 token 类型。它比 ERC-1155 更极简, 适合项目希望在一个合约内管理多个 FT 型商品券。

示例:

contract: BrandVoucherBook
id 1: 1 瓶可乐券
id 2: 1 瓶零度可乐券
id 3: 1 瓶雪碧券
id 4: 1 杯咖啡券

特点:

一个合约管理多个同质化权益。

合约更轻, 适合 L2 上大规模发行。

比一个 SKU 一个 ERC-20 更方便统一管理。

生态支持仍在增长, 但 OpenZeppelin 已有 ERC-6909 相关实现和接口。

适用:

一个品牌有大量 SKU。

一个电商店铺有多种可兑换商品。

一个游戏、会员、服务平台需要多种权益 token。

5.3 Profile C: ERC-1155 Multi Voucher

ERC-1155 是成熟的多 token 标准, 一个 tokenId 可以是同质化、半同质化或非同质化资产。它的生态和 NFT 平台支持更成熟, 适合作为兼容层。

特点:

一个合约管理多个 tokenId。

每个 tokenId 可以代表一个 SKU、批次、地区或有效期。

适合半同质化权益: 同一批券内部同质, 不同批次条款不同。

对 NFT 市场展示较友好。

适用:

有批次、有效期、地域差异的提货券。

需要兼容 NFT marketplace 或已有 ERC-1155 工具链的商户。

5.4 Profile D: NFT Voucher

NFT 只用于确实需要唯一性的场景。

适用:

1 个 token = 1 台唯一编号设备。

1 个 token = 1 张固定座位门票。

1 个 token = 1 件收藏联名商品。

1 个 token = 1 个保修合同或服务合同。

NFT 可以带来收藏价值, 但不应成为默认模型。

5.5 默认推荐

MVP 推荐顺序:

第一版: ERC-20 FT Voucher, decimals = 0, 支持 BURN 和 COLLECT

第二版: ERC-6909 Multi-FT Voucher

第三版: ERC-1155 兼容层

第四版: NFT / SFT 收藏与唯一权益扩展

6. 生命周期: 核销不等于销毁

传统电子券通常是:

发券 -> 用户使用 -> 券失效 -> 数据库标记已核销

链上 token 给了更丰富的选择。RedeemLoop 把「核销」和「结算动作」拆开。

6.1 BURN: 核销即销毁

用户持有 1 token

用户兑换商品

token 被 burn

总供应量减少 1

适合:

限量活动券。

一次性服务券。

预售提货券。

需要最终消耗供应量的商品券。

优点:

简单。

防重用清晰。

统计方便。

缺点:

token 不能再次分发。

如果厂商希望总量恒定, burn 会浪费可循环资产。

6.2 COLLECT: 核销即收回

用户持有 1 token

用户兑换商品

token 转入 Merchant Vault

厂商后续可再次出售、空投、分发

总供应量不变

适合:

可长期循环的标准消费品。

全国或全球通用兑换券。

品牌长期营销券。

厂商希望控制总量但反复使用的权益凭证。

优点:

总量恒定。

厂商可永续运营。

token 有真实流通闭环。

用户可自由转让, 厂商可在兑换后回收。

缺点:

需要更严格的金库和门店结算设计。

需要防止门店虚假收券或库存欺诈。

需要区分「被收回但未清算」和「已可重新分发」。

6.3 LOCK_AND_FULFILL: 先锁定, 后结算

对于需要物流交付的电商订单, 不建议用户提交订单时立即 burn 或 collect。正确流程是:

用户发起兑换

系统锁定 token

电商创建 0 元订单

厂商确认库存和收货信息

商品发货

履约完成

按发行条款 burn 或 collect

如果订单失败:

地址不可达 / 库存不足 / 风控失败 / 用户取消
token 解锁回用户
或换发其他 token
或退款

6.4 REISSUE: 回收后再分发

当 token 进入 Merchant Vault 后, 它不是死亡资产。厂商可以通过新的 Campaign 再分发:

Merchant Vault -> 广告渠道 -> 用户钱包
Merchant Vault -> 直播间红包 -> 用户钱包
Merchant Vault -> 会员福利 -> 用户钱包
Merchant Vault -> 分销商 -> 门店用户

6.5 推荐状态机

CREATED
-> ACTIVE
-> HELD_BY_USER
-> PRESENTED_FOR_REDEMPTION
-> ACCEPTED_BY_MERCHANT
-> SETTLED_BURNED
-> or SETTLED_COLLECTED
-> or LOCKED_PENDING_FULFILLMENT
-> FULFILLED
-> REISSUABLE
-> REDISTRIBUTED

异常状态:

EXPIRED
FROZEN
RECALLED
FAILED_REDEMPTION
REFUNDED
REPLACED
QUARANTINED

6.6 Quarantine / 回收隔离期

对于 collect 模式, 建议支持可选隔离期:

用户兑换 -> token 进入 Merchant Vault -> quarantine 24 小时 -> 可重新分发

用于防止:

门店误核销后立即再次流出。

订单未完全确认就重复分发。

欺诈性刷券。

跨渠道结算未完成。

7. 架构总览

RedeemLoop 分为链上协议、链下商业桥、一键植入 SDK 和收银验券端。



7.1 链上层

链上负责保存最小且不可抵赖的事实:

token 余额。

供应量上限。

厂商金库。

权益条款 hash。

核销模式。

门店终端授权。

redemption event。

收回或销毁动作。

链上不保存:

收货地址。

电话。

姓名。

订单明细中的隐私字段。

用户画像。

7.2 链下层

链下负责商业系统集成:

电商库存检查。

0 元订单创建。

POS 收银集成。

店员权限。

物流状态。

售后退款。

广告活动归因。

发券活动管理。

7.3 一键植入层

RedeemLoop 的成败在于植入体验。厂商不应该为了发行提货券而重写官网。

理想接入方式:

```
<script src="https://cdn.redeemloop.org/widget/v0/redeemloop.js"></script>
<div
  data-rdl-widget="claim"
  data-rdl-merchant="coca-cola"
  data-rdl-campaign="summer-2026"
  data-rdl-token="0xVoucherToken"
  data-rdl-chain-id="8453">
</div>
```

React 接入:

```
import { RedeemLoopProvider, ClaimButton, RedeemButton, VoucherBalance } from '@redeemloop/react';

export default function ProductPage() {
  return (
    <RedeemLoopProvider chainId={8453} merchantId="coca-cola">
      <VoucherBalance token="0xVoucherToken" />
      <ClaimButton campaignId="summer-2026" />
    </RedeemLoopProvider>
  );
}
```

```
        <RedeemButton sku="coke-bottle" mode="collect" />
    </RedeemLoopProvider>
  );
}
```

实体店 QR:

```
redeemloop://redeem?
chainId=8453&
token=0xVoucherToken&
amount=1&
storeId=tokyo-store-001&
terminalId=pos-07&
nonce=0x...
```

8. 一键代码植入: 官、商、小程序、直播、告渠道

8.1 厂商官网

官网需要四类组件:

ClaimButton: 领取或空投。

BuyButton: 购买提货券。

RedeemButton: 使用提货券兑换商品。

VoucherWallet: 展示该品牌相关 token。

目标体验:

复制一段代码 -> 粘贴到商品页 -> 配置 merchantId、campaignId、tokenAddress -> 上线

8.2 电商店铺

电商插件需要做两件事:

发行: 用户购买提货券后 mint 或 transfer token 给用户。

兑换: 用户 checkout 时用 token 支付该商品 line item, 创建 0 元订单。

关键原则:

不是抵扣金额, 而是该 line item 的完整权益已由 token 覆盖。

8.3 小程序

小程序可以通过 H5 钱包、MPC 钱包、账户抽象或托管钱包接入。RedeemLoop SDK 应该提供:

claim link。

redeem link。

QR scan。

后端签名 API。

用户 token 余额查询。

8.4 直播带货

直播间最适合分发可流转 FT Voucher:

主播口播 -> 屏幕 QR -> 用户 claim -> token 到钱包 -> 可转让或兑换

直播活动可以配置:

限量数量。

每个钱包最多领取数量。

时间窗口。

是否需要观看证明或渠道签名。

token 是否来自新发行或 Merchant Vault 回收库存。

8.5 广告分发渠道

广告渠道可以把 token 当作真实权益的获客媒介:

广告点击 -> claim intent -> 用户领取 token -> 到门店兑换 -> 链上 event 归因 -> 渠道结算

这比普通优惠券更强, 因为用户拿到的是完整商品权益, 不是「满减」或「积分」。

9. 电商与实体店验券

9.1 电商验券

电商 checkout 流程:

1. 用户选择商品。
2. 插件检查购物车 line item 是否匹配 voucher terms。
3. 插件读取用户 wallet 余额。
4. 用户签名 RedeemAuthorization。
5. CommerceBridge 调用 RedemptionRouter。
6. token 锁定、销毁或转入 Merchant Vault。
7. 电商创建 0 元订单。
8. 订单状态写入链下数据库, hash 上链或写入 event。

9.2 实体店验券

实体店流程:

1. 店员在 POS 选择可兑换商品。
2. POS 生成 redeem QR。
3. 用户扫码确认。
4. 用户钱包签名授权门店收取 1 个 token。
5. POS/relay 提交链上交易。
6. token 进入门店或厂商 vault，或被 burn。
7. POS 显示核销成功。
8. 门店交付商品。

9.3 为什么需要签名授权

实体店用户不应该为了换一瓶饮料而持有 gas。推荐使用 EIP-712 类型化签名：

用户签名：我同意 `storeId=tokyo-store-001` 在 5 分钟内收取 1 个 COKE1 token 用于兑换。

门店提交：`collectWithAuthorization(signature)`

签名字段必须包含：

`chainId`。

`voucherToken`。

`user`。

`merchantId`。

`storeId`。

`terminalId`。

`amount`。

`redemptionMode`。

`nonce`。

`deadline`。

`termsHash`。

9.4 离线与弱网

完全离线的链上验券不现实，但可以提供弱网模式：

POS 先验证最近缓存余额。

用户签名离线授权。

POS 在恢复网络后提交。

对高风险商品不允许弱网交付。

对低价商品可配置门店信用额度。

10. Coca-Cola 型永续流转案例

假设可口可乐发行一种全球通用饮料券：

Token: COKE1

类型: ERC-20 FT

Decimals: 0

供应量: 100,000,000

增发: 禁止

权益: 1 COKE1 = 兑换 1 瓶任意合格可口可乐公司饮品

核销: COLLECT

有效期: 长期有效或由条款定义

流转: 用户之间可自由转账

兑换地点: 授权门店、自动售货机、电商渠道

使用过程:

1. 厂商把 1,000,000 个 COKE1 发给夏季活动。
2. 用户 Alice 在直播间领取 3 个。
3. Alice 转给 Bob 1 个。
4. Bob 到便利店兑换一瓶可乐。
5. 便利店 POS 收取 Bob 的 1 个 COKE1。
6. token 进入 Japan Regional Merchant Vault。
7. 便利店获得厂商或渠道结算。
8. 厂商把回收的 token 再发给下一次广告活动。
9. 总供应量始终是 100,000,000。

这种模型的价值在于:

token 不是一次性消耗品。

厂商可以建立一个长期流通的商品权益网络。

用户之间转让自然发生, 协议不需要自己做二级市场。

门店收券后 token 回流, 厂商可以继续运营。

需要解决的问题:

不同国家商品规格不同, 条款必须定义清楚。

门店如何向厂商结算, 需要链下清算系统。

食品安全、地区限制、税务、消费者权益法规需要本地化。

若 token 可公开交易, 必须避免宣传为投资品。

如果长期有效, 可能触发预付凭证或类似监管要求, 需要法律评估。

11. 链上合约设计

11.1 合约模块

VoucherFactory

创建 ERC20Voucher、ERC6909Voucher、ERC1155Voucher

TermsRegistry

保存 termsHash、termsURI、merchantId、skuId、region、redemptionMode

MerchantRegistry

保存厂商身份、域名、vault、签名者、合规配置

StoreTerminalRegistry

保存被授权门店和 POS 终端

RedemptionRouter

处理 burn、collect、lock、fulfill、cancel

MerchantVault

接收回收 token, 管理二次分发和隔离期

CampaignDistributor

从厂商库存或 vault 向用户发放 token

11.2 RedemptionMode

```
enum RedemptionMode {  
    BURN,  
    COLLECT,  
    LOCK_AND_BURN,  
    LOCK_AND_COLLECT  
}
```

11.3 RedeemAuthorization

```
struct RedeemAuthorization {  
    address user;  
    address voucherToken;  
    uint256 tokenId;        // ERC-20 使用 0, ERC-6909/1155 使用 id  
    uint256 amount;  
    bytes32 merchantId;  
    bytes32 storeId;  
    bytes32 terminalId;  
    bytes32 termsHash;  
    uint8 redemptionMode;  
    uint256 nonce;  
    uint256 deadline;  
}
```

11.4 关键事件

```
event VoucherIssued(  
    address indexed voucherToken,
```

```

        address indexed to,
        uint256 indexed tokenId,
        uint256 amount,
        bytes32 campaignId
    );

    event VoucherRedeemed(
        bytes32 indexed redemptionId,
        address indexed user,
        address indexed voucherToken,
        uint256 tokenId,
        uint256 amount,
        uint8 redemptionMode,
        address settlementTarget,
        bytes32 storeId,
        bytes32 termsHash
    );

    event VoucherReissued(
        address indexed voucherToken,
        address indexed fromVault,
        address indexed to,
        uint256 tokenId,
        uint256 amount,
        bytes32 campaignId
    );

```

11.5 ERC-20 Voucher 必须具备的扩展

MVP 的 ERC-20 Voucher 应包含:

decimals() = 0。

maxSupply。

sealMinting()。

termsHash。

merchantId。

merchantVault。

redemptionMode。

collectWithAuthorization()。

burnWithAuthorization()。

pause()。

freezeRedemption() 可选。

recoverMistakenTokens() 受限。

11.6 供应量策略

RedeemLoop 支持三种供应策略:

FIXED_SEALED: 到达 maxSupply 或管理员 seal 后永不增发。

CAPPED_MINTABLE: 可在 cap 内继续发。

DYNAMIC_BACKED: 根据库存、保证金或生产承诺动态限制可售数量。

对「可口可乐型永续券」推荐:

FIXED_SEALED + COLLECT + MerchantVault redistributor

11.7 转让策略

协议层不默认禁止转让。可选策略:

OPEN_TRANSFER: 自由转账。

MERCHANT_ONLY_TRANSFER: 只能用户与厂商/门店之间转移。

ALLOWLIST_TRANSFER: 只能在合规名单内转让。

NON_TRANSFERABLE: 不可转让, 仅兑换。

你的目标场景应默认采用:

OPEN_TRANSFER

因为 RedeemLoop 要原生构建在现有加密货币基础设施之上, 用户二次流转应尽可能交给现有钱包、转账和市场。

12. 链下服务设计

12.1 Merchant Console

厂商后台功能:

创建兑换券。

选择 token profile。

设置商品权益条款。

设置核销模式: burn 或 collect。

设置总供应量和是否 seal。

设置门店、POS、渠道权限。

生成一键植入代码。

创建直播或广告发券活动。

查看兑换和回收统计。

从 Merchant Vault 再分发 token。

12.2 CommerceBridge

CommerceBridge 连接链上 voucher 和电商系统。

功能:

SKU 映射。

库存检查。

0 元订单创建。

发货状态同步。

退款、取消、换券。

订单 hash 写入事件。

12.3 POS Verifier

POS Verifier 提供:

门店登录。

商品选择。

QR 生成。

签名验证。

链上提交。

弱网缓存。

核销结果回调。

12.4 Campaign Distributor

分发系统支持:

新铸造发放。

从 Merchant Vault 回收库存发放。

白名单 claim。

直播 QR claim。

广告链接 claim。

邀请码 claim。

API 发券。

12.5 Indexer

Indexer 监听链上事件并写入数据库:

VoucherIssued。

VoucherRedeemed。

VoucherReissued。

Transfer。

VaultReceived。

VaultReleased。

TermsUpdated。

StoreAuthorized。

13. 数据模型与事件模型

13.1 核心表

merchants

id, legal_name, display_name, domain, admin_wallet, default_vault, status

voucher_classes

id, merchant_id, chain_id, token_address, token_profile, symbol,
decimals, max_supply, supply_policy, redemption_mode, terms_hash, terms_uri

campaigns

id, merchant_id, voucher_class_id, source_vault, distribution_type,
start_at, end_at, per_wallet_limit, status

stores

id, merchant_id, region, address_hash, status

terminals

id, store_id, public_key, operator_wallet, status

redemptions

id, merchant_id, store_id, terminal_id, user_wallet, token_address,
token_id, amount, mode, status, tx_hash, order_ref_hash, created_at

vault_movements

```
id, vault, token_address, token_id, amount, movement_type,  
campaign_id, redemption_id, tx_hash
```

13.2 订单隐私

收货地址、电话、姓名、身份证件、配送备注等不应上链。链上只保留:

```
orderRefHash = keccak256(orderId + merchantId + salt)  
shippingCommitment = keccak256(addressPayload + salt)
```

13.3 可观测指标

厂商后台需要展示:

已发行数量。

用户持有数量。

Merchant Vault 持有数量。

已 burn 数量。

已 collect 数量。

兑换地区分布。

渠道分发效果。

token 周转率。

库存覆盖率。

欺诈警报。

14. 用户体验

14.1 用户不应理解复杂链上概念

用户只需要看到:

你有 3 张可乐兑换券

可兑换 3 瓶可乐

去兑换 / 转给朋友 / 保存到钱包

不应该让用户面对:

gas。

approve。

tokenId。

bridge。

RPC。

私钥助记词。

14.2 账户模式

RedeemLoop 支持三种账户：

用户自有钱包: MetaMask、Rabby、Coinbase Wallet 等。

Embedded Wallet: 邮箱、手机号、Passkey 创建。

Smart Account: ERC-4337 或其他账户抽象方案, 由 Paymaster 赞助 gas。

14.3 一键领取

领取体验:

点击领取 -> 登录或连接钱包 -> 签名或自动发送 -> token 到账

14.4 一键兑换

兑换体验:

扫码 -> 显示商品和门店 -> 确认兑换 -> 成功 -> 拿到商品

14.5 转让体验

由于 token 原生是 FT, 用户可以直接用现有钱包转账。RedeemLoop 可以提供辅助按钮:

转给朋友

生成收款码

复制钱包地址

但不需要自己建设完整二级交易市场。

15. 安全设计

15.1 防重复核销

每个签名授权必须有 nonce。

每个授权必须有 deadline。

每个 redemptionId 只能使用一次。

StoreTerminal 必须在注册表中有效。

TermsHash 必须与当前 token class 匹配。

15.2 防签名重放

签名域必须包含：

chainId。

verifyingContract。

voucherToken。

merchantId。

storeId。

terminalId。

nonce。

deadline。

15.3 门店权限

门店和 POS 终端必须被厂商授权。不同门店可配置不同权限：

CAN_COLLECT
CAN_BURN
CAN_LOCK
CAN_REFUND
CAN_REISSUE
DAILY_LIMIT
SKU_SCOPE
REGION_SCOPE

15.4 厂商金库安全

Merchant Vault 应支持：

多签管理。

分角色授权。

单日转出限额。

回收隔离期。

只允许向 CampaignDistributor 发放。

异常暂停。

15.5 智能合约安全

MVP 必须包含:

Foundry 单元测试。

Fuzz test。

Slither 静态扫描。

重入保护。

签名域测试。

nonce 消耗测试。

pause 和 recovery 测试。

collect/burn 两种模式测试。

ERC-20 decimals=0 测试。

16. 积分分支: 只能作为旁路, 不进入主体验

RedeemLoop 不把积分作为核心资产。积分可以存在, 但只能作为旁路模块:

积分 -> 换 Voucher Token

积分 -> 升级权益

积分 -> 获得优先领取资格

积分 -> 获得白名单

不能做成:

积分抵扣 10 元, 用户还要补 90 元

积分汇率复杂变化

积分可投机交易

积分替代完整提货券

积分模块应隔离在:

packages/loyalty-adapters/

接口:

```
interface LoyaltyAdapter {
  getBalance(user: string, merchantId: string): Promise<bigint>;
  earn(event: LoyaltyEvent): Promise<void>;
  burnForVoucher(input: BurnForVoucherInput): Promise<MintOrTransferIntent>;
}
```

17. 合规与商业边界

RedeemLoop 应坚持以下对外表述:

这是商品或服务兑换权益凭证, 不是投资产品。

需要避免:

承诺升值。

宣传收益。

做价格排行榜。

把券叫做币。

把二级市场作为主要卖点。

对不可履约商品过量发行。

需要明确:

厂商是否担保。

平台是否担保。

是否可退款。

是否可转让。

是否有有效期。

哪些地区可兑换。

缺货如何处理。

token 被回收后是否会再次分发。

在某些司法区, 长期有效、可转让、可购买、可兑换商品的 token 可能涉及预付凭证、礼品卡、电子货币、消费者权益、税务或虚拟资产规则。项目发布前必须做本地法律评估。

18. MVP 开发路线

Phase 0: 原型

目标:

用户领取 ERC-20 Voucher -> 实体店扫码 -> 用户签名 -> 门店收回 token -> 显示核销成功

范围:

一个 ERC-20 Voucher。

decimals = 0。

fixed max supply。

collectWithAuthorization。

Merchant Vault。

一个 Web Widget。

一个 POS Verifier 页面。

一个 Indexer。

不做:

多链桥。

二级市场。

NFT。

复杂积分。

复杂 KYC。

Phase 1: 电商可用版

增加:

BuyButton。

RedeemButton。

CommerceBridge。

0 元订单。

Lock-and-fulfill。

退款和取消。

Campaign Distributor。

Phase 2: 多商品与多渠道

增加:

ERC-6909 VoucherBook。

ERC-1155 兼容层。

小程序 SDK。

直播 QR 发放。

广告渠道归因。

多门店结算。

Phase 3: 企业与合规扩展

增加:

ERC-7943 风格 RWA 合规接口。

ERC-3643 风格 permissioned transfer 适配器。

KYC/allowlist 插件。

区域限制。

保证金和履约证明。

多签和审计。

19. Codex 施工原则

让 Codex 开发时, 不要一次性要求实现所有功能。应按可测试闭环拆分任务。

第一闭环:

ERC20Voucher + MerchantVault + collectWithAuthorization + POS Web Demo

第二闭环:

Claim Widget + Campaign Distributor + Indexer

第三闭环:

CommerceBridge + 0 元订单 Demo + lock-and-fulfill

第四闭环:

ERC6909VoucherBook + 多 SKU + SDK 抽象

每个闭环必须有:

合约测试。

API 测试。

前端可点击 demo。

本地 docker-compose。

README 步骤。

安全注意事项。

20. 参考资料

ERC-20 Token Standard: <https://eips.ethereum.org/EIPS/eip-20>

ERC-1155 Multi Token Standard: <https://eips.ethereum.org/EIPS/eip-1155>

ERC-6909 Minimal Multi-Token Interface: <https://eips.ethereum.org/EIPS/eip-6909>

OpenZeppelin Contracts: <https://docs.openzeppelin.com/contracts>

EIP-712 Typed Structured Data Hashing and Signing: <https://eips.ethereum.org/EIPS/eip-712>

ERC-3009 Transfer With Authorization: <https://eips.ethereum.org/EIPS/eip-3009>

ERC-4337 Account Abstraction: <https://eips.ethereum.org/EIPS/eip-4337>

ERC-4337 Paymasters: <https://docs.erc4337.io/paymasters/index.html>

ERC-7943 Universal RWA Interface: <https://eips.ethereum.org/EIPS/eip-7943>

ERC-3643 Permissioned Token / T-REX: <https://eips.ethereum.org/EIPS/eip-3643>

一句话总结

RedeemLoop 是一个 FT-first 的 RWA 提货券协议: 厂商可以一键发行和植入「1 token = 1 个完整商品或服务权益」的链上兑换券, 用户可在现有加密基础设施中持有和流转, 电商与实体店可轻松验券, 核销时既可以销毁, 也可以收回到厂商金库后再次分发, 从而形成总量恒定、可永续运营的商品权益流通闭环。